

# Methodology: A Sequential LSTM Portfolio Allocator with a Learned Risk-Attenuation Overlay

Architecture, Mathematical Formulation, and Implementation Reference

Repository: `mlforecasting`

Modules: `models/portfolio_lstm.py`, `models/risk_lstm.py`, `api/server.py`

July 31, 2026

## Abstract

This document describes, in full mathematical and implementation detail, an FX portfolio allocation system consisting of two independently-trained neural networks applied *sequentially*: (1) **PortfolioLSTM**, which maps a rolling window of multi-asset FX log returns (optionally augmented with interest-rate carry and vol-normalized momentum features) to a portfolio weight vector, followed by a volatility-targeting rescaling step; and (2) **RiskLSTM**, an optional overlay trained *on top of* the frozen portfolio decisions, which learns a per-asset attenuation factor in  $[\text{max\_attenuation}, 1]$  that scales down positions when an asset's recent (marginal) and portfolio-wide (cross-sectional) risk statistics look unstable. Every architectural choice is presented alongside its rationale, the exact tensor shapes at each stage, the governing mathematical expression, the source file and line range implementing it, and – where relevant – the optional variants that can be toggled via configuration. Two full architecture diagrams and a data-flow diagram are included. The document closes with a complete configuration reference and a summary table of every design decision and its motivation.

## Contents

|          |                                                                                |           |
|----------|--------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Notation and Conventions</b>                                                | <b>3</b>  |
| <b>2</b> | <b>System Overview</b>                                                         | <b>3</b>  |
| <b>3</b> | <b>Data Pipeline and Feature Engineering</b>                                   | <b>4</b>  |
| 3.1      | Price loading and log returns . . . . .                                        | 4         |
| 3.2      | Feature engineering: carry and vol-normalized momentum . . . . .               | 4         |
| 3.3      | Rolling window construction . . . . .                                          | 6         |
| 3.4      | Standardization . . . . .                                                      | 6         |
| <b>4</b> | <b>Chronological Train / Validation / Test Split</b>                           | <b>7</b>  |
| <b>5</b> | <b>PortfolioLSTM</b>                                                           | <b>7</b>  |
| 5.1      | Encoder A: concatenated input . . . . .                                        | 8         |
| 5.2      | Encoder B: shared per-asset encoder with cross-asset attention . . . . .       | 8         |
| 5.3      | Temporal pooling: last hidden state vs. attention pooling . . . . .            | 9         |
| 5.4      | Weight schemes . . . . .                                                       | 10        |
| 5.5      | Recurrent-policy feedback: feeding the previous weight into the head . . . . . | 10        |
| 5.6      | NoisyNet: parameter-space exploration noise . . . . .                          | 11        |
| 5.7      | The CASH pseudo-asset . . . . .                                                | 12        |
| <b>6</b> | <b>Volatility Targeting</b>                                                    | <b>12</b> |
| 6.1      | Three covariance estimators . . . . .                                          | 13        |
| 6.2      | The leverage cap . . . . .                                                     | 14        |

|           |                                                                            |           |
|-----------|----------------------------------------------------------------------------|-----------|
| <b>7</b>  | <b>Training Objectives</b>                                                 | <b>14</b> |
| 7.1       | Rolling-window Sortino-style ratio ("sharpe", default) . . . . .           | 14        |
| 7.2       | Kelly criterion ("kelly") . . . . .                                        | 15        |
| 7.3       | Mean-CVaR ("cvar") . . . . .                                               | 15        |
| <b>8</b>  | <b>Transaction Costs</b>                                                   | <b>15</b> |
| <b>9</b>  | <b>Portfolio Training Loop</b>                                             | <b>15</b> |
| <b>10</b> | <b>RiskLSTM: Risk-Attenuation Overlay</b>                                  | <b>16</b> |
| 10.1      | Marginal risk features: rolling moments of the weighted PnL path . . . . . | 17        |
| 10.2      | Cross-sectional risk features (optional) . . . . .                         | 17        |
| 10.3      | Building RiskLSTM's input sequences . . . . .                              | 18        |
| 10.4      | RiskLSTM architecture and forward pass . . . . .                           | 19        |
| 10.5      | Risk training loop . . . . .                                               | 20        |
| <b>11</b> | <b>Inverse-Volatility Benchmark</b>                                        | <b>20</b> |
| <b>12</b> | <b>Ensembling: Multi-Seed Restarts</b>                                     | <b>20</b> |
| <b>13</b> | <b>Configuration Reference</b>                                             | <b>21</b> |
| <b>14</b> | <b>Summary of Architectural Decisions</b>                                  | <b>22</b> |
| <b>15</b> | <b>Repository Map</b>                                                      | <b>23</b> |

## 1 Notation and Conventions

Throughout this document, matrix and tensor dimensions are written using the following symbols, matching the variable names used in code wherever possible:

| Symbol | Code name                          | Meaning                                                                         |
|--------|------------------------------------|---------------------------------------------------------------------------------|
| $B$    | <code>batch / n_samples</code>     | Number of decision samples in a batch (a full training split, in full-batch)    |
| $L$    | <code>lookback</code>              | Sequence length: number of trailing days in one input window                    |
| $N$    | <code>n_assets</code>              | Number of tradable assets (FX pairs, plus CASH if enabled)                      |
| $C$    | <code>n_channels</code>            | Feature channels per asset (1 = raw return only; > 1 with carry/vol-normalized) |
| $H$    | <code>hidden_size</code>           | LSTM hidden state width                                                         |
| $W$    | <code>rolling_window (risk)</code> | Trailing window (days) for RiskLSTM’s rolling statistics                        |
| $K$    | <code>risk_rolling_window-1</code> | Samples dropped from each split when building RiskLSTM sequences                |

A raw daily log-return vector is  $r_t \in \mathbb{R}^N$ , so an input window for one decision sample is  $X_i \in \mathbb{R}^{L \times NC}$  (flattened as  $L \times NC$  in the asset-major layout described in §3.2), a full split’s worth of input windows is a rank-3 tensor  $X \in \mathbb{R}^{B \times L \times NC}$ , and the portfolio weight vector for sample  $i$  is  $w_i \in \mathbb{R}^N$ . Superscript “raw” denotes the pre-standardization / pre-vol-targeting quantity; a hat ( $\hat{\cdot}$ ) denotes a model estimate. All neural network code is PyTorch; `nn.LSTM` is always instantiated with `batch_first=True`, so its own input/output convention is (batch, time, features), matching the convention used throughout this document.

Every feature described below is a **configuration flag with a documented default** – Section 13 tabulates the full set. Features whose default is *off* are marked with an **Optional component** box; enabling them changes the effective network input/output width, which is always noted explicitly.

## 2 System Overview

The system trains two networks *sequentially and independently* – never jointly. Figure 1 shows the full data flow, from raw prices to reported PnL.

- Stage 1. Data & features** (§3): daily close prices → log returns → optional carry / vol-normalized-momentum channels → rolling windows → chronological train/validation/test split → per-feature standardization.
- Stage 2. PortfolioLSTM** (§5): maps each window to a raw weight vector  $\hat{w}_i^{\text{raw}}$ , trained end-to-end (full-batch, over the whole training period at once) against a chosen risk-adjusted objective (§7), net of transaction costs (§8).
- Stage 3. Volatility targeting** (§6):  $\hat{w}_i^{\text{raw}}$  is rescaled so the portfolio’s *estimated* annualized volatility matches a target (e.g. 20%), using one of three covariance estimators.
- Stage 4. RiskLSTM overlay** (§10, optional): frozen PortfolioLSTM output is fed, together with rolling per-asset risk statistics and (optionally) cross-sectional market-structure statistics, to a second network that outputs a per-asset attenuation factor in  $[\text{max\_attenuation}, 1]$ .
- Stage 5. Reporting** (§11): realized PnL, Sharpe ratios, and an inverse-volatility benchmark (vol-matched to the model) are computed on all three splits.

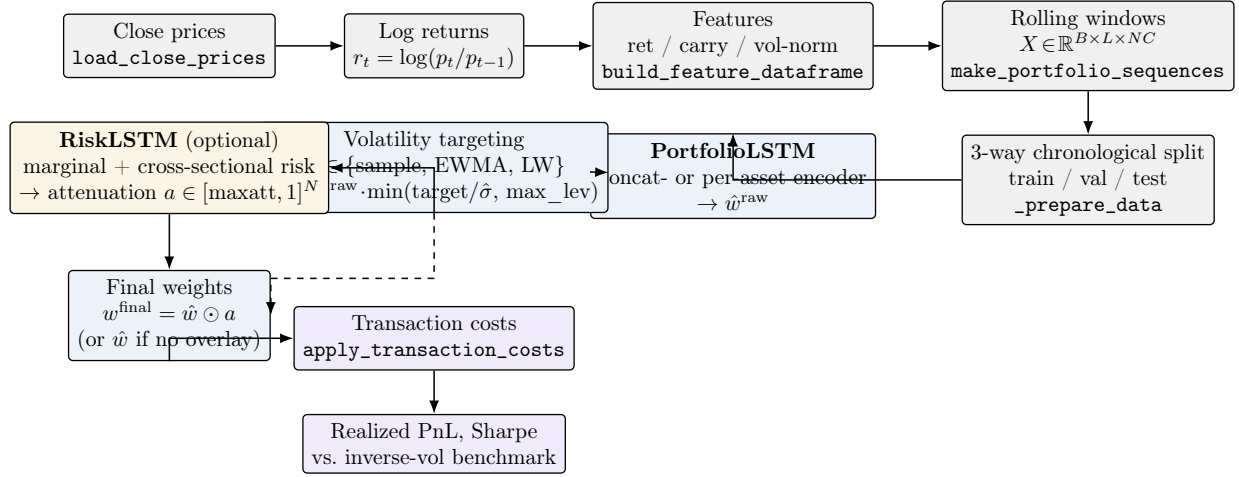


Figure 1: End-to-end pipeline. Dashed arrow: when the risk overlay is disabled, vol-targeted weights flow directly to the final-weights stage with  $a \equiv 1$ . Grey = data/feature engineering, blue = PortfolioLSTM & volatility targeting, amber = RiskLSTM overlay, violet = reporting.

**Rationale.** The two networks are trained *sequentially*, not jointly: PortfolioLSTM is trained (or loaded) to completion first, with *no notion* that a risk overlay might follow; RiskLSTM is then trained on the frozen, already-decided portfolio purely to improve its realized Sharpe ratio by learning *when to size it down*. This separation keeps each network’s optimization problem well-posed and interpretable: PortfolioLSTM’s gradient signal is entirely about *which assets to hold*, while RiskLSTM’s gradient signal is entirely about *how much of the already-chosen book to actually take*. A joint objective would conflate the two and make it impossible to attribute a Sharpe improvement to either network individually.

### 3 Data Pipeline and Feature Engineering

#### 3.1 Price loading and log returns

```
1 def to_log_returns(prices: pd.DataFrame) -> pd.DataFrame:
2     """Convert a price DataFrame to log returns: r_t = log(p_t / p_{t-1})."""
3     return np.log(prices / prices.shift(1)).dropna()
```

Listing 1: Log-return conversion.

models/portfolio\_lstm.py:189-191

For close price  $p_t$ , the log return is

$$r_t = \log\left(\frac{p_t}{p_{t-1}}\right), \quad r_t \in \mathbb{R}^{N_{\text{real}}} \quad (1)$$

where  $N_{\text{real}}$  is the number of *real* (non-CASH) tradable pairs. `load_close_prices` (lines 165–192) fetches an inner-joined (dropna’d), wide price DataFrame – one column per pair, indexed by date – via `data/db.py`, downloading and upserting any missing pair through `FXDownloader` on demand.

**Rationale.** Log returns (rather than simple returns) are used because they are additive across time ( $\sum_t r_t = \log(p_T/p_0)$ ) and because standard deviation/variance formulas for volatility targeting and risk-factor decomposition assume additivity of the underlying process.

#### 3.2 Feature engineering: carry and vol-normalized momentum

**Optional component (opt-in, default off).** Two additional per-asset feature channels can be enabled independently: **FX carry** (`use_carry`) and **vol-normalized momentum** at one or more horizons (`vol_horizons`). Both default to *off* ( $C = 1$ : raw return only).

**Carry.** For a pair with base currency  $b$  and quote currency  $q$  (e.g. EURUSD  $\Rightarrow b = \text{EUR}, q = \text{USD}$ ), carry is the interest-rate differential:

$$\text{carry}_t^{(b,q)} = \frac{\rho_t^{(b)} - \rho_t^{(q)}}{100} \quad (2)$$

where  $\rho_t^{(c)}$  is currency  $c$ 's short-term policy rate (percentage points, from FRED, monthly frequency) at time  $t$ , divided by 100 to bring it to the same order of magnitude as a log return. Reindexed onto the daily returns index with **forward-fill only** (never back-filled) so that no future rate print ever leaks into an earlier day; any day before the first available print is set to a neutral 0.

```
1 diff = (rates[base] - rates[quote]) / 100.0 # percentage points -> decimal
2 carry[pair] = diff.reindex(carry.index, method="ffill").fillna(0.0)
```

Listing 2: Carry construction (forward-fill only – no look-ahead).

models/portfolio\_lstm.py:207-241 (function load\_carry)

**Vol-normalized momentum.** For horizon  $h$  (days),

$$\text{volnorm}_t^{(h)} = \frac{\sum_{k=0}^{h-1} r_{t-k}}{\sigma_h(r)_t + \epsilon}, \quad \sigma_h(r)_t = \text{std}(r_{t-h+1}, \dots, r_t), \quad \epsilon = 10^{-8} \quad (3)$$

i.e. the trailing  $h$ -day cumulative return divided by the trailing  $h$ -day realized volatility – a standard risk-adjusted momentum/reversal signal, comparable across assets and volatility regimes.

```
1 cum_return = returns.rolling(h).sum()
2 vol = returns.rolling(h).std() + eps
3 normalized = cum_return / vol
```

Listing 3: Vol-normalized return at horizon  $h$ .

models/portfolio\_lstm.py:244-265 (function vol\_normalized\_returns)

**Asset-major channel layout.** When either optional feature is enabled,  $C = 1 + \mathbb{I}[\text{use\_carry}] + |\text{vol\_horizons}|$ , and the feature tensor is laid out **asset-major**: all of one asset's  $C$  channels are contiguous, then the next asset's:

$$\text{features} = \left[ \underbrace{r^{(1)}, \text{carry}^{(1)}, \text{volnorm}^{(1,h_1)}, \dots}_{\text{asset 1, } C \text{ channels}}, \underbrace{r^{(2)}, \dots}_{\text{asset 2}}, \dots \right] \in \mathbb{R}^{T \times NC} \quad (4)$$

with **channel 0 of every asset's block always the raw return** – this invariant is relied on later to recover a raw-returns-only view (channel-stride slicing  $X[:, :, 0 :: C]$ ) for volatility targeting and for RiskLSTM's input reconstruction, without carrying a separate copy of the raw series.

```
1 def build_feature_dataframe(returns, pairs, use_carry, vol_horizons, years):
2     n_channels = 1 + int(use_carry) + len(vol_horizons)
3     ...
4     for pair in pairs:
5         features[f"{pair}_ret"] = returns[pair]
6         if use_carry:
7             features[f"{pair}_carry"] = carry[pair]
```

```

8     for h in vol_horizons:
9         features[f"{pair}_vol{h}"] = vol_norm[f"{pair}_vol{h}"]
10    features = features.ffill().fillna(0.0) # never back-filled
11    assert features.shape[1] == n_channels * len(pairs)

```

Listing 4: Asset-major feature assembly.

models/portfolio\_lstm.py:268-306

**Rationale.** Asset-major (rather than channel-major, i.e. all returns then all carries) ordering is chosen deliberately so that a *contiguous slice* of the feature axis always belongs to exactly one asset – this is exactly what the per-asset shared encoder (§5.2) needs to slice out each asset’s own window with a single reshape, and what the raw-return-recovery stride trick needs.

### 3.3 Rolling window construction

```

1 def make_portfolio_sequences(returns, lookback, feature_returns=None):
2     values = returns.to_numpy(dtype=np.float32) # (T, N)
3     feature_values = feature_returns.to_numpy(...) if feature_returns is not None else values
4     for start in range(max(len(values) - lookback, 0)):
5         day_t = start + lookback - 1
6         X.append(feature_values[start : day_t + 1]) # (L, N*C)
7         next_returns.append(values[day_t + 1]) # (N,)
8         dates.append(index[day_t + 1])

```

Listing 5: Building lookback windows and next-day targets.

models/portfolio\_lstm.py:314-364

For each decision day  $t$  (indexed by sample  $i$ ), the window

$$X_i = [\text{features}_{t-L+1}, \dots, \text{features}_t] \in \mathbb{R}^{L \times NC} \quad (5)$$

is the model’s input, and the realized *next-day* return  $r_{t+1} \in \mathbb{R}^N$  (raw, un-featurized – always the plain log return, regardless of  $C$ ) is the target used to compute realized PnL:  $\text{pnl}_i = w_i^\top r_{t+1}$ .

**Tensor shapes.**  $X \in \mathbb{R}^{B \times L \times NC}$  where  $B = T - L$  ( $T$  = number of usable days of history);  $\text{next\_returns} \in \mathbb{R}^{B \times N}$ .

### 3.4 Standardization

Per-feature mean/std are fit **only on the training split** and reused, unchanged, for validation and test (and, when loading a saved model, restored from its checkpoint rather than refit):

$$\mu = \text{mean}_{(B,L)}(X^{\text{train,raw}}), \quad \sigma = \text{std}_{(B,L)}(X^{\text{train,raw}}) + 10^{-8}, \quad \mu, \sigma \in \mathbb{R}^{NC} \quad (6)$$

$$X^{\text{split}} = \frac{X^{\text{split,raw}} - \mu}{\sigma}, \quad \text{split} \in \{\text{train, val, test}\} \quad (7)$$

models/portfolio\_lstm.py:2150-2158 (in `_prepare_data`)

**Rationale.** Standardizing with train-only statistics (never refit on validation/test) prevents any information about the held-out splits’ scale from leaking into what the network sees as “normal.” The  $+10^{-8}$  epsilon safely handles a zero-variance channel – e.g. a constant-return CASH column, or carry for a currency pair FRED has no rate data for – without a division-by-zero.

## 4 Chronological Train / Validation / Test Split

```

1 test_frac = getattr(args, "test_frac", 0.0) or 0.0
2 n_total = len(X)
3 n_test = int(n_total * test_frac)
4 n_remaining = n_total - n_test
5 n_train = int(n_remaining * args.train_frac)
6
7 X_full_train_raw = X[:n_train]
8 X_full_val_raw = X[n_train:n_remaining]
9 X_full_test_raw = X[n_remaining:]

```

Listing 6: 3-way chronological split.

models/portfolio\_lstm.py:2131-2148 (\_prepare\_data)

The split is strictly **chronological**: the most recent `test_frac` fraction of *all* samples is carved off *first* as the test set, and `train_frac` is then applied to whatever remains:

$$n_{\text{test}} = \lfloor n_{\text{total}} \cdot \text{test\_frac} \rfloor \quad (8)$$

$$n_{\text{remaining}} = n_{\text{total}} - n_{\text{test}} \quad (9)$$

$$n_{\text{train}} = \lfloor n_{\text{remaining}} \cdot \text{train\_frac} \rfloor \quad (10)$$

$$n_{\text{val}} = n_{\text{remaining}} - n_{\text{train}} \quad (11)$$

With defaults `train_frac = 0.8`, `test_frac = 0.1`: the oldest 72% of history is train, the next 18% is validation, and the most recent 10% is test. `test_frac = 0` recovers the original 2-way split exactly ( $n_{\text{test}} = 0$ ).

**Rationale.** The test set is held out from every training and model-selection decision – unlike validation, whose Sharpe ratio *does* drive best-epoch checkpoint selection (§9), the test split’s returns are computed every epoch purely for live *display* and never influence which epoch’s weights get kept. This is what makes test performance a genuinely unbiased read on generalization: a model that overfits validation by sheer trial-and-error across many configurations would still be caught by test, since test was never available to that selection process at all. Carving test off *before* train/val (rather than, say, taking a random subset) preserves the temporal ordering assumed by a real backtest – the model never trains on data from *after* the days it is ultimately evaluated on.

## 5 PortfolioLSTM

PortfolioLSTM maps an input window  $X_i \in \mathbb{R}^{L \times NC}$  to a portfolio weight vector  $\hat{w}_i \in \mathbb{R}^N$ . Two encoder architectures are available (`encoder_type`), sharing everything downstream of the encoder (temporal pooling, weight scheme, previous-weight feedback).

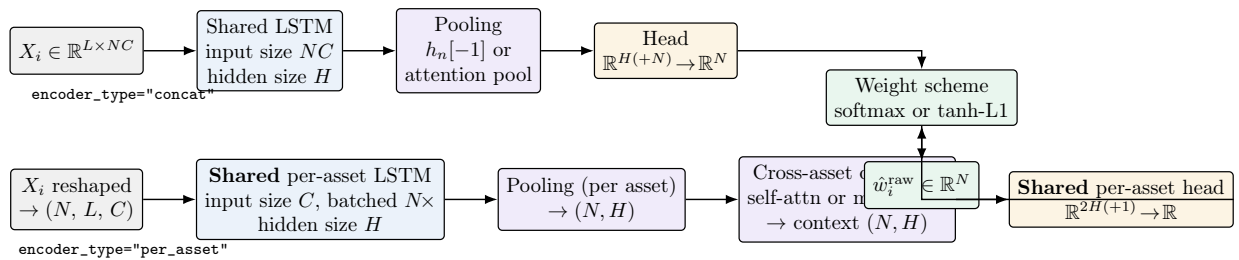


Figure 2: PortfolioLSTM’s two interchangeable encoders. Both feed the same downstream weight scheme; `prev_weight` feedback (§5.5) concatenates into the head’s input in both branches (the whole vector  $w_{i-1} \in \mathbb{R}^N$  for “concat”, only the asset’s own scalar  $w_{i-1}^{(k)}$  for “per\_asset”).

## 5.1 Encoder A: concatenated input

```

1 self.lstm = nn.LSTM(input_size=n_assets * n_channels, hidden_size=hidden_size,
2                     num_layers=num_layers, batch_first=True)
3 head_input_size = hidden_size + n_assets if use_prev_weight else hidden_size
4 self.head = NoisyLinear(head_input_size, n_assets) if noisy_head else nn.Linear(
    head_input_size, n_assets)

```

Listing 7: Concatenated-input encoder (default).

models/portfolio\_lstm.py:592-607 (PortfolioLSTM.\_\_init\_\_)

One LSTM consumes every asset's features concatenated into a single per-timestep vector:

$$\text{LSTM} : \mathbb{R}^{B \times L \times NC} \rightarrow (\text{lstm\_out} \in \mathbb{R}^{B \times L \times H}, h_n \in \mathbb{R}^{1 \times B \times H}) \quad (12)$$

followed by pooling over time (§5.3) to a single  $\mathbb{R}^{B \times H}$  summary, then the head (a single `Linear`/`NoisyLinear` layer):

$$\hat{\text{logits}}_i = \text{Head}([\text{pooled}_i; w_{i-1}]) \in \mathbb{R}^N \quad (\text{concatenation omitted when } \text{use\_prev\_weight} = \text{False}) \quad (13)$$

**Rationale.** This is the simplest, fastest architecture and the historical default. Its weakness: every learned weight is tied to a *fixed asset order and count* – the LSTM's input weight matrix has one column block per specific asset, and the head's output has one fixed neuron per specific asset. Changing the universe (adding/removing/reordering pairs) requires retraining from scratch; this motivated the per-asset encoder below.

## 5.2 Encoder B: shared per-asset encoder with cross-asset attention

```

1 self.asset_lstm = nn.LSTM(input_size=n_channels, hidden_size=hidden_size,
2                           num_layers=num_layers, batch_first=True)
3 if asset_combiner == "attention":
4     self.asset_attn = nn.MultiheadAttention(embed_dim=hidden_size, num_heads=n_attn_heads,
5                                             batch_first=True)
6 head_input_size = 2 * hidden_size + (1 if use_prev_weight else 0)
7 self.head = NoisyLinear(head_input_size, 1) if noisy_head else nn.Linear(head_input_size, 1)

```

Listing 8: Per-asset encoder + cross-asset combiner.

models/portfolio\_lstm.py:608-633

```

1 batch, lookback, _ = x.shape
2 x_per_asset = x.view(batch, lookback, self.n_assets, self.n_channels).permute(0, 2, 1, 3)
3 x_flat = x_per_asset.reshape(batch * self.n_assets, lookback, self.n_channels)
4 lstm_out, (h_n, _) = self.asset_lstm(x_flat)
5 pooled = self.temporal_pool(lstm_out) if self.pooling == "attention" else h_n[-1]
6 h = self.dropout(pooled).view(batch, self.n_assets, self.hidden_size) # (B, N, H)
7
8 if self.asset_combiner == "attention":
9     context, _ = self.asset_attn(h, h, h, need_weights=False) # (B, N, H)
10 else:
11     context = h.mean(dim=1, keepdim=True).expand(-1, self.n_assets, -1) # (B, N, H)
12
13 combined = torch.cat([h, context], dim=-1) # (B, N, 2H)
14 if self.use_prev_weight:
15     combined = torch.cat([combined, prev_weight.unsqueeze(-1)], dim=-1) # (B, N, 2H+1)
16 logits = self.head(combined).squeeze(-1) # (B, N)

```

Listing 9: Per-asset encoder forward pass.



models/portfolio\_lstm.py:669-693

**One LSTM, with parameters shared across every asset**, is applied independently to each asset's own  $C$ -channel window, by folding the asset dimension into the batch dimension:

$$X_i \in \mathbb{R}^{L \times NC} \rightarrow \tilde{X}_i \in \mathbb{R}^{N \times L \times C} \rightarrow \tilde{X}_i^{\text{flat}} \in \mathbb{R}^{(BN) \times L \times C} \quad (14)$$

$$\text{LSTM}_{\text{shared}} : \mathbb{R}^{(BN) \times L \times C} \rightarrow h \in \mathbb{R}^{B \times N \times H} \quad (\text{after pooling and reshape}) \quad (15)$$

Every asset's own hidden state  $h_i^{(k)} \in \mathbb{R}^H$  then either **attends to** or is **mean-pooled with** every other asset's hidden state, producing a cross-asset context  $c_i^{(k)} \in \mathbb{R}^H$  (**asset\_combiner**):

$$\text{attention: } c_i = \text{MultiheadAttention}(Q=h_i, K=h_i, V=h_i) \in \mathbb{R}^{N \times H} \quad (16)$$

$$\text{mean: } c_i^{(k)} = \frac{1}{N} \sum_{j=1}^N h_i^{(j)} \quad \forall k \quad (17)$$

A **single, shared** per-asset head then maps each asset's own  $[h_i^{(k)}; c_i^{(k)}] \in \mathbb{R}^{2H}$  (optionally +1 for  $w_{i-1}^{(k)}$ , that asset's *own* previous weight, a scalar) to *one* logit:

$$\hat{\text{logit}}_i^{(k)} = \text{Head}_{\text{shared}}([h_i^{(k)}; c_i^{(k)}; w_{i-1}^{(k)}]) \in \mathbb{R} \quad (18)$$

stacked over  $k = 1, \dots, N$  to form  $\hat{\text{logits}}_i \in \mathbb{R}^N$ , exactly as in the concatenated encoder.

**Rationale.** Because *every* learned weight (the shared LSTM, the attention/mean combiner, the shared head) is reused across assets rather than tied to one specific asset index, this architecture is **permutation-equivariant**: reordering the input pairs reorders the output weights identically (verified empirically – §??). It is also, in principle, **universe-size invariant**: nothing in the forward pass depends on a fixed value of  $N$  except bookkeeping (reshape targets, zero-initialization width), so the same trained weights could in principle score a different *number* of assets than trained on (not currently exposed as a supported workflow, but architecturally unconstrained). Self-attention across assets additionally lets the network learn genuine *pairwise* relationships (e.g. two correlated pairs, or a natural hedge) rather than only a symmetric market-wide average, which is what the simpler "mean" combiner provides as a cheaper, more stable alternative when there are few assets or little training history.

**Tensor shapes.**  $x \in \mathbb{R}^{B \times L \times NC} \rightarrow \text{view} \rightarrow (B, L, N, C) \rightarrow \text{permute} \rightarrow (B, N, L, C) \rightarrow \text{reshape} \rightarrow (BN, L, C)$ ; asset-LSTM output pooled to  $(BN, H) \rightarrow \text{view} \rightarrow (B, N, H)$ ; context  $(B, N, H)$ ; **combined**  $(B, N, 2H)$  or  $(B, N, 2H+1)$ ; **logits** after `squeeze(-1)`:  $(B, N)$ .

### 5.3 Temporal pooling: last hidden state vs. attention pooling

**Optional component (opt-in, default off).** `pooling="last"` (default) uses  $h_n[-1]$ , the LSTM's final hidden state, as the whole window's summary. `pooling="attention"` instead uses a learned attention pool over *every* timestep's hidden state. Applies identically to both PortfolioLSTM encoders and to RiskLSTM.

```
1 class TemporalAttentionPool(nn.Module):
2     def __init__(self, hidden_size):
3         self.score = nn.Sequential(nn.Linear(hidden_size, hidden_size), nn.Tanh(),
4                                     nn.Linear(hidden_size, 1))
5     def forward(self, lstm_out): # (*, T, H) -> (*, H)
6         scores = self.score(lstm_out) # (*, T, 1)
7         weights = torch.softmax(scores, dim=-2) # (*, T, 1)
8         return (weights * lstm_out).sum(dim=-2) # (*, H)
```

Listing 10: Learned attention pooling over the full LSTM output sequence.

models/portfolio\_lstm.py:458-492

For an LSTM output sequence  $h_1, \dots, h_T \in \mathbb{R}^H$  (the standard additive/Bahdanau attention pool):

$$s_t = v^\top \tanh(Wh_t), \quad \alpha_t = \frac{\exp(s_t)}{\sum_{\tau=1}^T \exp(s_\tau)}, \quad \text{pooled} = \sum_{t=1}^T \alpha_t h_t \quad (19)$$

where  $W \in \mathbb{R}^{H \times H}$ ,  $v \in \mathbb{R}^H$  are learned (`self.score`'s two `Linear` layers). The leading dimension is generic: for PortfolioLSTM's per-asset encoder, pooling is applied to a  $(BN, T, H)$  tensor (each asset-sample row pooled independently); for the concatenated encoder and for RiskLSTM, to a  $(B, T, H)$  tensor.

**Rationale.**  $h_n[-1]$  forces the LSTM to compress everything relevant about the whole window into whatever it happens to be carrying at the very *last* timestep – a real limitation for longer lookbacks where the most decision-relevant days (e.g. a vol spike) might sit in the middle rather than at the end. Attention pooling instead lets the network learn, per timestep, how relevant that day's hidden state is to the current decision, and combines all of them into one summary vector.

## 5.4 Weight schemes

$$\text{softmax (long-only):} \quad \hat{w}_i = \text{softmax}(\text{logits}_i), \quad \hat{w}_i \succeq 0, \quad \mathbf{1}^\top \hat{w}_i = 1 \quad (20)$$

$$\text{tanh\_norm (long/short):} \quad \tilde{w}_i = \tanh(\text{logits}_i), \quad \hat{w}_i = \frac{\tilde{w}_i}{\|\tilde{w}_i\|_1 + \epsilon}, \quad \|\hat{w}_i\|_1 = 1 \quad (21)$$

models/portfolio\_lstm.py:635-642 (`_weights_from_logits`)

**Rationale.** `softmax` guarantees a valid long-only book (non-negative, fully invested) by construction – convenient when short positions are undesirable or unavailable. `tanh_norm` allows each asset's weight to be positive *or* negative (a genuine long/short book), normalized so the book's gross exposure ( $\sum_k |\hat{w}_i^{(k)}|$ ) is exactly 1, matching softmax's "fully invested" property in gross-exposure terms while allowing sign flexibility.

## 5.5 Recurrent-policy feedback: feeding the previous weight into the head

**Optional component (opt-in, default off).** `use_prev_weight` (default `False`) concatenates the previously realized weight  $w_{i-1}$  into the head's input, following the classic Moody & Saffell recurrent-policy trick, so the network can reason about whether a rebalance is worth its transaction cost rather than deciding each window as if starting from flat.

```

1 def forward_sequence(self, X, initial_weight=None):
2     if not self.use_prev_weight:
3         return self.forward(X) # ordinary vectorized batch call
4
5     n = X.shape[0]
6     prev = initial_weight.detach() if initial_weight is not None else torch.zeros(self.n_assets, ...)
7     freeze = self.noisy_head and self.training
8     if freeze:
9         self.head.resample_noise(); self.head.freeze_noise(True)
10    try:
11        weights = []
12        for i in range(n):
13            w = self.forward(X[i:i+1], prev.unsqueeze(0)).squeeze(0)
14            weights.append(w)
15            prev = w.detach() # DETACHED - not full BPTT
16        return torch.stack(weights, dim=0)
17    finally:
18        if freeze: self.head.freeze_noise(False)

```

Listing 11: The sequential recurrent-policy loop (`forward_sequence`).

models/portfolio\_lstm.py:695-755

When `use_prev_weight=False`, sample  $i$ 's decision is independent of sample  $i-1$ 's, so  $B$  samples are processed in one ordinary vectorized batch call – identical speed to a model without this feature. When `use_prev_weight=True`, sample  $i$ 's *input* genuinely depends on sample  $i-1$ 's *output*, forcing a sequential Python loop over  $i = 1, \dots, B$ :

$$w_i = f_\theta(X_i, w_{i-1}), \quad w_{i-1} \leftarrow \text{sg}(w_{i-1}) \quad (22)$$

where `sg(·)` (“stop-gradient”, i.e. `.detach()`) means  $w_{i-1}$  is **detached** before being fed forward as input to step  $i$ : gradients flow normally through *each step's own local computation* (all steps share the same LSTM/head parameters  $\theta$ , and a loss summed over the whole path trains  $\theta$  using every step's signal), but this is **not** full backpropagation-through-time – unrolling gradients through hundreds or thousands of sequential steps would be both computationally intractable and numerically unstable. This is a deliberate simplification: the network observes its own last action as part of its state (a genuine recurrent policy), but is not asked to reason about how today's decision causally shapes next week's gradient.

**Implementation note. NoisyLinear in-place-mutation fix.** `NoisyLinear` resamples its noise buffers (`weight_epsilon`, `bias_epsilon`) *in place* on every `forward()` call while training (§5.6). The sequential loop above calls `self.forward()` once *per day* in the training period – without intervention, every one of those  $B$  calls would overwrite the *same* buffer in place, corrupting an earlier step's saved value before the eventual single `loss.backward()` over the whole period could use it (a real `RuntimeError`: “one of the variables needed for gradient computation has been modified by an in-place operation”). The fix: `resample_noise()` is called **once** before the loop (one fresh noise draw for this whole sequential pass, matching what one ordinary `forward()` call would get), then `freeze_noise(True)` holds it fixed for the loop's duration, and `freeze_noise(False)` in a **finally** block restores normal per-call resampling afterward. **Design constraint this reveals:** enabling `noisy_head` together with `use_prev_weight` forces one shared noise draw per training epoch (rather than per day) – a strictly coarser noise schedule than the non-recurrent path, but the only one compatible with a single backward pass over the whole trajectory.

**Rationale.** `initial_weight` lets a caller continue a position *across* a split boundary (e.g. validation continues from training's last decided weight, test continues from validation's) rather than restarting from flat at each boundary – a real portfolio does not reset to cash at the end of a backtest period.

## 5.6 NoisyNet: parameter-space exploration noise

**Optional component (opt-in, default off).** `noisy_head` (default `False`) replaces the head's plain `nn.Linear` with `NoisyLinear` (Fortunato et al., 2017), whose weight/bias are themselves stochastic.

```

1 class NoisyLinear(nn.Module):
2     def __init__(self, in_features, out_features, sigma_init=0.5):
3         self.weight_mu = nn.Parameter(torch.empty(out_features, in_features))
4         self.weight_sigma = nn.Parameter(torch.empty(out_features, in_features))
5         self.bias_mu = nn.Parameter(torch.empty(out_features))
6         self.bias_sigma = nn.Parameter(torch.empty(out_features))
7         self.register_buffer("weight_epsilon", torch.empty(out_features, in_features))
8         self.register_buffer("bias_epsilon", torch.empty(out_features))
9
10    def forward(self, x):
11        if self.training:
12            if not self._frozen: self._reset_noise() # epsilon ~ N(0,1), in place

```

```

13     weight = self.weight_mu + self.weight_sigma * self.weight_epsilon
14     bias = self.bias_mu + self.bias_sigma * self.bias_epsilon
15     else:
16         weight, bias = self.weight_mu, self.bias_mu # deterministic at inference
17     return F.linear(x, weight, bias)

```

Listing 12: NoisyLinear: independent Gaussian parameter noise.

models/portfolio\_lstm.py:371-455

$$W = \mu_W + \sigma_W \odot \varepsilon_W, \quad \varepsilon_W \sim \mathcal{N}(0, 1)^{d_{\text{out}} \times d_{\text{in}}}, \quad b = \mu_b + \sigma_b \odot \varepsilon_b, \quad \varepsilon_b \sim \mathcal{N}(0, 1)^{d_{\text{out}}} \quad (23)$$

with  $\sigma_W, \sigma_b$  initialized to  $\text{sigma\_init}/\sqrt{d_{\text{in}}}$  ( $\text{sigma\_init}=0.5$  default) and *learned* via gradient descent like any other parameter, so the model can shrink the noise magnitude over training if it stops helping. At inference (`model.eval()`),  $\mu$  alone is used – deterministic, mirroring how dropout also disables itself outside training.

**Rationale.** This is the “noisy cell” analogue of `noise_std` (which perturbs the *input* window instead, §9): rather than the model seeing a slightly different input each epoch, it must produce a good Sharpe ratio under a slightly different version of its *own decision boundary* each epoch, making it harder to lock onto one razor-sharp weight configuration that happens to maximize Sharpe on the exact training sequence but does not generalize. The two noise sources compose (both may be enabled simultaneously) rather than substitute for each other. RiskLSTM applies the *same* noisy-linear treatment to **both** of its head layers (§10), for full NoisyNet coverage of its decision path.

## 5.7 The CASH pseudo-asset

**Optional component (opt-in, default off).** `has_cash` (default `False`) adds one more holdable “asset” with a constant daily return `cash_return` (default 0.0).

```

1 if has_cash:
2     returns = returns.copy()
3     returns["CASH"] = cash_return
4     pairs = real_pairs + ["CASH"]

```

Listing 13: CASH is a data-layer pseudo-pair, not a model-layer special case.

models/portfolio\_lstm.py:1936-1956 (`_prepare_data`)

**Rationale.** CASH is implemented *not* as a model-level output-dimension special case, but as a **data-level pseudo-pair** appended to `pairs` and to the `returns` DataFrame *before* windowing. This means it is treated as “just another asset” everywhere downstream (vol-targeting, transaction costs, per-asset reporting, the risk overlay) with **zero special-casing** required elsewhere: a constant-return column has exactly zero realized variance, which the covariance-based vol-targeting formula (§6) already handles correctly (a risk-free asset contributes no variance, exactly as it should), and  $N \leftarrow N + 1$  propagates automatically through every shape computed from `len(pairs)`.

## 6 Volatility Targeting

PortfolioLSTM’s raw weights  $\hat{w}_i^{\text{raw}}$  are rescaled so the portfolio’s *estimated* annualized volatility matches a target (e.g. 20%):

$$\hat{\sigma}_i = \sqrt{\hat{w}_i^{\text{raw}\top} \hat{\Sigma}_i \hat{w}_i^{\text{raw}}} \cdot \sqrt{252}, \quad \hat{w}_i = \hat{w}_i^{\text{raw}} \cdot \min\left(\frac{\text{target\_vol}}{\max(\hat{\sigma}_i, 10^{-8})}, \text{max\_leverage}\right) \quad (24)$$

models/portfolio\_lstm.py:1384-1453 (`portfolio_volatility, scale_weights_to_target_vol`)

**Tensor shapes.**  $\hat{\Sigma}_i \in \mathbb{R}^{N \times N}$  per sample;  $\hat{w}_i^{\text{raw}} \in \mathbb{R}^N$ ;  $\hat{\sigma}_i \in \mathbb{R}$  (scalar per sample); batched over  $B$  samples via `torch.einsum`.

## 6.1 Three covariance estimators

**Optional component (opt-in, default off).** `covariance_estimator` (default "sample") selects among three estimators of  $\hat{\Sigma}_i$  from the same-sample raw-return window  $X_i^{\text{raw}} \in \mathbb{R}^{L \times N}$  (channel-0-only, i.e. raw returns, never carry or vol-normalized channels).

### (a) Sample covariance (default).

$$\bar{r}_i = \frac{1}{L} \sum_{t=1}^L X_i^{(t)}, \quad \hat{\Sigma}_i^{\text{sample}} = \frac{1}{L-1} \sum_{t=1}^L (X_i^{(t)} - \bar{r}_i)(X_i^{(t)} - \bar{r}_i)^\top \quad (25)$$

**(b) EWMA (RiskMetrics-style).** Day  $t$  ( $t=0$  oldest,  $t=L-1$  newest) is weighted  $\propto \lambda^{L-1-t}$ , renormalized to sum to 1 (a finite window truncates the infinite geometric series):

$$\omega_t = (1 - \lambda) \lambda^{L-1-t} / \sum_{\tau} (1 - \lambda) \lambda^{L-1-\tau}, \quad \bar{r}_i^{\text{ewma}} = \sum_t \omega_t X_i^{(t)} \quad (26)$$

$$\hat{\Sigma}_i^{\text{ewma}} = \sum_{t=1}^L \omega_t (X_i^{(t)} - \bar{r}_i^{\text{ewma}})(X_i^{(t)} - \bar{r}_i^{\text{ewma}})^\top \quad (27)$$

with  $\lambda = \text{ewma\_lambda}$  (default 0.94, the RiskMetrics daily default,  $\approx 11$ -day half-life).

**(c) Ledoit-Wolf shrinkage (Ledoit & Wolf, 2004).** Shrinks the sample covariance toward a well-conditioned scaled-identity target  $F = \bar{\mu}I$ ,  $\bar{\mu} = \frac{1}{N} \text{tr}(\hat{\Sigma}^{\text{sample}})$ , using the analytically optimal (not fixed/hand-tuned) shrinkage intensity:

$$\hat{\pi} = \frac{1}{L-1} \sum_{t=1}^L \|X_i^{(t)} X_i^{(t)\top} - \hat{\Sigma}_i^{\text{sample}}\|_F^2 \quad (\text{estimator noise}) \quad (28)$$

$$\hat{\gamma} = \|\hat{\Sigma}_i^{\text{sample}} - F\|_F^2 \quad (\text{target misspecification}) \quad (29)$$

$$\delta^* = \text{clip}\left(\frac{\hat{\pi}}{\max(\hat{\gamma}, 10^{-12})}, 0, 1\right) \quad (30)$$

$$\hat{\Sigma}_i^{\text{LW}} = \delta^* F + (1 - \delta^*) \hat{\Sigma}_i^{\text{sample}} \quad (31)$$

models/portfolio\_lstm.py:1297-1381 (estimate\_covariance)

```
1 outer = torch.einsum("bti,btj->btij", centered, centered) # per-day outer products
2 deviation = outer - sample_cov.unsqueeze(1)
3 pi_hat = (deviation ** 2).sum(dim=(-1, -2)).mean(dim=1) / max(lookback - 1, 1)
4 gamma_hat = ((sample_cov - target) ** 2).sum(dim=(-1, -2))
5 delta = (pi_hat / gamma_hat.clamp(min=1e-12)).clamp(min=0.0, max=1.0)
6 return delta * target + (1.0 - delta) * sample_cov
```

Listing 14: Ledoit-Wolf shrinkage intensity (batched, vectorized).

**Rationale.** The plain sample covariance over a short window ( $L=30$  days by default) is noisy: a handful of coincidentally calm or coincidentally correlated days can swing the whole estimate, and if  $L \leq N$  it can even be exactly singular. EWMA is a strictly cheaper improvement – weighting recent days more lets the estimate react faster to a genuine volatility regime change while still using the whole window for stability. Ledoit-Wolf goes further: it inflates small eigenvalues toward a well-conditioned target using an intensity *derived*, not guessed, from how noisy the sample estimator itself is – directly addressing the sample estimator’s worst failure mode for vol targeting (see `max_leverage` below): a

near-singular covariance produces a spuriously tiny estimated risk and hence an enormous leverage scale-up, right before a normal-sized move can wipe out the position. Empirically verified (unit test): on a rank-1 (perfectly collinear) synthetic window, the sample estimator's smallest eigenvalue is  $\approx 0$  while Ledoit-Wolf's is safely bounded away from zero.

## 6.2 The leverage cap

```
1 if target_vol == 0: return weights
2 vol = portfolio_volatility(window_returns, weights, ...).clamp(min=1e-8)
3 scale = (target_vol / vol).clamp(max=max_leverage)
4 return weights * scale.unsqueeze(-1)
```

Listing 15: `max_leverage` caps the scale factor itself.

models/portfolio\_lstm.py:1409-1453

**Rationale.** Without a cap on *the scale factor itself* (not merely a numerical-stability clamp buried inside the volatility estimate), a window whose ESTIMATED covariance happens to be near-zero can be scaled up by 1000× or more (confirmed empirically on realistic synthetic data), and that single sample can then realize a catastrophic ( $< -100\%$ ) return the moment the actual next-day move is not equally calm. This is especially severe for the "kelly" training objective:  $\log(1 + r)$  has a singularity at  $r = -1$ , and the safety clamp used there produces an *exact zero gradient* for any return at or below it – the model would get no corrective signal from precisely the sample that most needs one. Capping leverage at a sane multiple (default 10×) bounds every sample's worst-case return and fixes this dead-gradient failure mode *at its source*, rather than papering over the symptom independently in each loss function.

## 7 Training Objectives

```
1 def compute_training_loss(portfolio_returns, objective, sharpe_window=60, cvar_alpha=0.95,
2   cvar_kappa=1.0):
3   if objective == "sharpe": return -rolling_window_ratio(portfolio_returns, window=
4     sharpe_window, downside=True)
5   if objective == "kelly": return kelly_loss(portfolio_returns)
6   if objective == "cvar": return mean_cvar_loss(portfolio_returns, alpha=cvar_alpha, kappa=
7     cvar_kappa)
```

Listing 16: Objective dispatcher.

models/portfolio\_lstm.py:1264-1290

### 7.1 Rolling-window Sortino-style ratio ("sharpe", default)

$$\text{ratio}_j = \frac{\bar{r}_j}{\sqrt{\frac{1}{w} \sum_{t \in W_j} \min(r_t, 0)^2 + \epsilon}} \cdot \sqrt{252}, \quad \mathcal{L}_{\text{sharpe}} = -\frac{1}{J} \sum_{j=1}^J \text{ratio}_j \quad (32)$$

where  $W_j$  ranges over all  $J = B - w + 1$  overlapping  $w$ -day sub-windows (`sharpe_window`, default 60) of the  $B$ -day training period, and the denominator is a **downside** deviation (only negative returns penalized – the Sortino variant) rather than the plain standard deviation. models/portfolio\_lstm.py:1131-1177 (rolling\_window\_ratio)

**Rationale.** Scoring the mean ratio over many overlapping sub-windows, rather than one ratio over the whole training period, prevents the model from finding one weight pattern that happens to nail the exact historical sequence end-to-end while behaving arbitrarily badly on any sub-period. Downside-only risk (Sortino) does not penalize *upside* volatility, which is a more appropriate risk definition for a return-seeking strategy than symmetric variance.



## 7.2 Kelly criterion ("kelly")

$$\mathcal{L}_{\text{kelly}} = -\mathbb{E}[\log(1 + r_t)] \approx -\frac{1}{B} \sum_{t=1}^B \log(\max(1 + r_t, \epsilon)) \quad (33)$$

models/portfolio\_lstm.py:1180-1202

**Rationale.** Maximizes expected *log*-wealth growth – the Kelly criterion – which is convex in the portfolio weights (no extra tunable risk-aversion weight needed) and targets long-run compounded growth rather than a single-period ratio. Its numerical fragility at  $r \rightarrow -1$  is exactly why `max_leverage` (§6) exists.

## 7.3 Mean-CVaR ("cvar")

$$\text{CVaR}_\alpha = \frac{1}{k} \sum_{j=1}^k \ell_{(j)}, \quad \ell_t = -r_t, \quad k = \max(1, \text{round}((1 - \alpha)B)) \quad (34)$$

(mean of the  $k$  worst realized losses –  $\ell_{(1)} \geq \dots \geq \ell_{(k)}$  are the top- $k$  losses via `torch.topk`, whose gradient flows through whichever days are currently the worst).

$$\mathcal{L}_{\text{cvar}} = - \underbrace{\left( \frac{1}{B} \sum_t r_t \right) \cdot 252}_{\text{annualized mean}} + \kappa \cdot \underbrace{\text{CVaR}_\alpha \cdot \sqrt{252}}_{\text{annualized CVaR}} \quad (35)$$

models/portfolio\_lstm.py:1205-1261 (cvar, mean\_cvar\_loss)

**Rationale.** This is the textbook convex portfolio objective (Rockafellar & Uryasev, 2000): both the negative mean (affine) and CVaR (convex) are convex in the weights, so their nonnegative combination is convex in weight-space – at the cost of `cvar_kappa`, a *fixed* (not learned) risk-aversion weight, since there is no way to combine two different quantities into one strictly convex scalar without such a weight (it plays the same structural role as mean-variance’s  $\lambda$ , applied to CVaR instead of variance). **Both terms are annualized before combining** – this was a deliberate fix: on raw daily FX returns the mean is tiny while CVaR (average of the worst days) is 10–30× larger in practice, so combining them *unannualized* with  $\kappa=1.0$  made the loss almost entirely about minimizing CVaR, with the mean term contributing well under 10% of the gradient signal – the model barely learned to seek return at all. Annualizing first (mean  $\times 252$ , CVaR  $\times \sqrt{252}$ , matching how Sharpe itself annualizes) brings both terms to a comparable order of magnitude, making  $\kappa=1.0$  a genuinely balanced default.

## 8 Transaction Costs

$$\text{turnover}_t = \begin{cases} \|w_0\|_1 & t = 0 \\ \|w_t - w_{t-1}\|_1 & t > 0 \end{cases}, \quad r_t^{\text{net}} = r_t - \frac{\text{transaction\_cost\_bps}}{10^4} \cdot \text{turnover}_t \quad (36)$$

models/portfolio\_lstm.py:1514-1573 (apply\_transaction\_costs, and the differentiable apply\_transaction\_costs\_torch)

**Rationale.** Costs are applied **net, inside the training loss itself** (via the differentiable `apply_transaction_costs_torch`) not only as a post-hoc reporting adjustment – so the model is trained to maximize its objective *net of* trading costs directly, learning to avoid abrupt, expensive day-to-day weight swings whenever they are not worth their cost (this is precisely what motivates `use_prev_weight`, §5.5: without seeing its own previous position, the model cannot reason about whether a rebalance is worth its cost at all). The day-0 cost ( $\|w_0\|_1$ ) charges the cost of putting the initial position on from flat.

## 9 Portfolio Training Loop

```

1  for epoch in range(1, epochs + 1):
2      optimizer.zero_grad()
3      noisy_X_train = X_train + randn_like(X_train) * noise_std if noise_std > 0 else X_train
4      raw_weights = model.forward_sequence(noisy_X_train) # (B_train, N)
5      weights = scale_weights_to_target_vol(raw_weights, X_train_raw, target_vol, ...)
6      portfolio_returns = (weights * next_returns_train).sum(dim=-1) # (B_train,)
7      portfolio_returns = apply_transaction_costs_torch(weights, portfolio_returns,
8          transaction_cost)
9      loss = compute_training_loss(portfolio_returns, objective, sharpe_window, cvar_alpha,
10          cvar_kappa)
11      loss.backward(); optimizer.step()
12
13  if track_val:
14      model.eval()
15      val_weights = scale_weights_to_target_vol(model.forward_sequence(X_val, initial_weight=
16          val_initial), ...)
17      val_sharpe = float(sharpe_ratio(val_returns)) # plain whole-period Sharpe
18      model.train()
19      if val_sharpe > best_val_sharpe:
20          best_val_sharpe = val_sharpe
21          best_state = copy.deepcopy(model.state_dict()) # checkpoint selection
22
23  # after the loop:
24  if track_val and best_state is not None:
25      model.load_state_dict(best_state) # restore best-VALIDATION-Sharpe epoch

```

Listing 17: `train_portfolio_model`: one full-batch epoch.

models/portfolio\_lstm.py:1580-1744

Training is **full-batch**: every epoch computes weights for the *entire* training period at once ( $B_{\text{train}}$  samples in parallel, or sequentially if `use_prev_weight`), because Sharpe/Sortino/CVaR are statistics of a whole return *distribution* and cannot be meaningfully evaluated on one sample at a time the way a per-sample loss (e.g. MSE) can.

**Rationale. Checkpoint selection by validation Sharpe** (the plain whole-period metric, *not* whatever **objective** is training against) rather than always keeping the last epoch directly targets an in-sample-great, out-of-sample-poor training curve: the run keeps whichever epoch’s weights actually generalized best, a standard early-stopping / checkpoint-selection use of a validation split. **Test-set tracking** (when `test_frac > 0`) computes the identical quantity on the test split every epoch too, but *purely for live display* – it never enters `best_state` selection, preserving test’s status as a genuinely unbiased read (§4). `noise_std` perturbs only the *standardized input*, not the clean window used for vol-targeting (that estimate must reflect real, not artificially noised, risk).

## 10 RiskLSTM: Risk-Attenuation Overlay

**Optional component (opt-in, default off).** The entire risk overlay (`risk_overlay`, default `False`) is optional. PortfolioLSTM is always trained (or loaded) to completion *first*, with no notion that an overlay might follow; RiskLSTM is then trained on the frozen result, purely to improve its realized Sharpe by learning *when to size it down*.



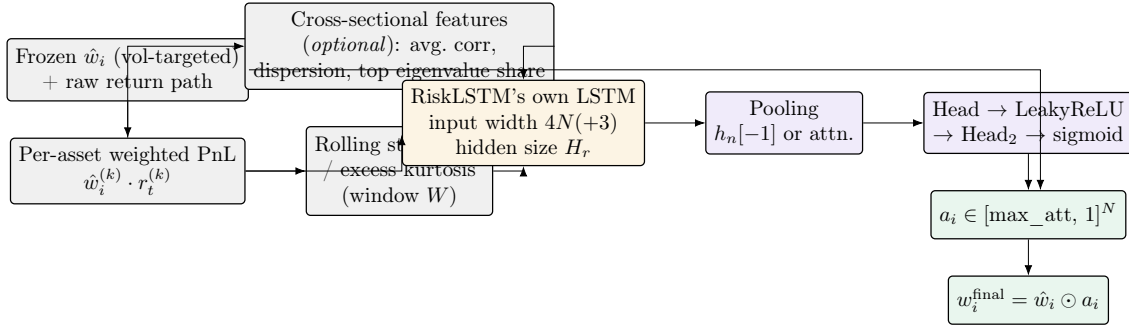


Figure 3: RiskLSTM: the input feature stack (per-asset weighted PnL, marginal rolling moments, and optional cross-sectional features) feeds its own LSTM; the frozen portfolio weight vector re-enters at the head as a conditioning input, exactly as in PortfolioLSTM’s `use_prev_weight`.

### 10.1 Marginal risk features: rolling moments of the weighted PnL path

```

1 def rolling_moments(x, window, eps=1e-6): # x: (B, L, N)
2     windows = x.unfold(dimension=1, size=window, step=1) # (B, T', N, window)
3     mean = windows.mean(dim=-1, keepdim=True)
4     centered = windows - mean
5     variance = (centered ** 2).mean(dim=-1)
6     std = torch.sqrt(variance + eps)
7     skewness = (centered ** 3).mean(dim=-1) / (std ** 3 + eps)
8     kurtosis = (centered ** 4).mean(dim=-1) / (std ** 4 + eps) - 3.0 # excess
9     return torch.cat([std, skewness, kurtosis], dim=-1) # (B, T', 3N)

```

Listing 18: Rolling std / skewness / excess kurtosis.

models/risk\_lstm.py:94-128

For each asset  $k$  and trailing  $W$ -day sub-window ending at  $t$ , on the **per-asset weighted PnL path**  $p_t^{(k)} = \hat{w}_t^{(k)} \cdot r_t^{(k)}$  (the raw return path, scaled by that sample’s OWN decision weight – i.e. statistics of the asset’s *actual exposure*, which vol-targeting may have leveraged up or `tanh_norm` may have flipped short – rather than of the asset’s own unlevered return):

$$\mu_t^{(k)} = \frac{1}{W} \sum_{\tau=t-W+1}^t p_\tau^{(k)}, \quad \sigma_t^{(k)} = \sqrt{\frac{1}{W} \sum_{\tau} (p_\tau^{(k)} - \mu_t^{(k)})^2 + \epsilon} \quad (37)$$

$$\text{skew}_t^{(k)} = \frac{\frac{1}{W} \sum_{\tau} (p_\tau^{(k)} - \mu_t^{(k)})^3}{(\sigma_t^{(k)})^3 + \epsilon}, \quad \text{kurt}_t^{(k)} = \frac{\frac{1}{W} \sum_{\tau} (p_\tau^{(k)} - \mu_t^{(k)})^4}{(\sigma_t^{(k)})^4 + \epsilon} - 3 \quad (38)$$

**Rationale.** Feeding these three moments *directly*, rather than hoping an LSTM rediscovers them from raw returns, is a more direct and sample-efficient way to teach a small network “is there a clear, stable trend or not”: std is realized volatility (the classic risk signal), skewness flags asymmetric tails (crash risk vs. melt-up), and excess kurtosis (0 for a Normal distribution) flags fat tails / regime instability. Using the *weighted* PnL (not the raw return) means these statistics reflect the asset’s actual current exposure, not its unlevered volatility – a highly-levered small position can be riskier in dollar terms than an unlevered large one.

### 10.2 Cross-sectional risk features (optional)

**Optional component (opt-in, default off).** `use_cross_sectional` (default `False`) appends 3 **portfolio-wide** (not per-asset) channels computed from the rolling cross-asset correlation structure

of the **raw, unweighted** return path (deliberately not the weighted path – a zero-weighted asset’s degenerate constant series would make correlation undefined).

```

1 def cross_sectional_features(x, window, eps=1e-6): # x: (B, T, N) RAW returns
2     windows = x.unfold(1, window, 1) # (B, T', N, window)
3     normalized = (windows - windows.mean(-1, keepdim=True)) / std # per-asset z-score
4     corr = torch.matmul(normalized, normalized.transpose(-1, -2)) / window # (B, T', N, N)
5
6     off_diag = corr * (1 - eye); n_pairs = N * (N - 1)
7     avg_corr = off_diag.sum(dim=(-1, -2)) / n_pairs
8     corr_dispersion = sqrt(((corr - avg_corr)**2 * (1-eye)).sum(dim=(-1, -2)) / n_pairs)
9     eigenvalues = torch.linalg.eigvalsh(corr) # ascending, real (symmetric)
10    top_eig_share = eigenvalues[..., -1] / eigenvalues.sum(dim=-1)
11    return torch.stack([avg_corr, corr_dispersion, top_eig_share], dim=-1) # (B, T', 3)

```

Listing 19: Cross-sectional correlation-structure features.

models/risk\_lstm.py:131-195

For the rolling  $N \times N$  correlation matrix  $\hat{\rho}_t$  (per-asset  $z$ -scored returns’ Gram matrix, so diagonal = 1 by construction):

$$\bar{\rho}_t = \frac{1}{N(N-1)} \sum_{k \neq j} \hat{\rho}_t^{(k,j)} \quad [\text{avg. pairwise correlation}] \quad (39)$$

$$s_t = \sqrt{\frac{1}{N(N-1)} \sum_{k \neq j} (\hat{\rho}_t^{(k,j)} - \bar{\rho}_t)^2} \quad [\text{correlation dispersion}] \quad (40)$$

$$\phi_t = \frac{\lambda_{\max}(\hat{\rho}_t)}{\sum_{m=1}^N \lambda_m(\hat{\rho}_t)} = \frac{\lambda_{\max}(\hat{\rho}_t)}{N} \quad [\text{top eigenvalue share}] \quad (41)$$

(the trace of a correlation matrix is always  $N$ , so the denominator of Eq. (41) is exactly  $N$ ;  $\phi_t \in [1/N, 1]$ : close to  $1/N$  means no dominant common factor, close to 1 means one shared risk driver is moving nearly everything). With  $N < 2$ , pairwise correlation is undefined and these three channels are returned as zero.

**Rationale.** This is the complement to the marginal moments above: an asset can look individually calm while the *whole book* is one bad day away from moving together – a regime the per-asset statistics alone cannot see. Average correlation and its dispersion distinguish a uniform one-factor market (high average, low dispersion) from a patchwork one (some pairs correlated, others not); the top-eigenvalue share is the standard PCA-based “how much day-to-day covariation is explained by one common factor” concentration measure. Computed on *raw*, not weighted, returns so the correlation structure reflects genuine market co-movement, independent of the current portfolio’s own (possibly zero) weights.

### 10.3 Building RiskLSTM’s input sequences

```

1 n_valid = n_samples - rolling_window + 1
2 full_series = torch.cat([X_raw[0], next_returns_raw], dim=0) # reconstruct continuous series
3 extended_len = lookback + rolling_window - 1
4 extended_windows = full_series.unfold(0, extended_len, 1)[:n_valid] # (n_valid, extended_len, N)
5
6 weighted = extended_windows * weights[:, None, :] # this sample's OWN decision weight
7 moments = rolling_moments(weighted, rolling_window) # (n_valid, lookback, 3N)
8 aligned_returns = weighted[:, rolling_window - 1:, :] # (n_valid, lookback, N)
9 features = torch.cat([aligned_returns, moments], dim=-1) # (n_valid, lookback, 4N)
10 if use_cross_sectional:
11     features = torch.cat([features, cross_sectional_features(extended_windows, rolling_window)], dim=-1)

```

Listing 20: `make_risk_sequences`: decoupling `rolling_window` from `lookback`.

models/risk\_lstm.py:198-303

To let `rolling_window` ( $W$ ) be sized *independently* of `lookback` ( $L$ ), the continuous raw-return series underlying  $X^{\text{raw}}$  is first reconstructed (the first window in full, then exactly the one new day each subsequent sample adds), then, for each sample, an **extended** slice of length  $L + W - 1$  (not just  $L$ ) is taken, weighted by that sample’s own decision weight, and *then* rolled with window  $W$  – producing exactly  $L$  output timesteps per sample regardless of how  $W$  relates to  $L$ . The first  $W - 1$  samples of a split lack enough leading context and are **dropped** (a small, fixed cost, independent of  $L$ ); `weights/next_returns` are returned sliced to match, so every downstream consumer (training targets, reporting, plots) reuses the identical alignment.

**Tensor shapes.** Final feature tensor:  $(n_{\text{valid}}, L, 4N [+3])$ , where  $n_{\text{valid}} = n_{\text{samples}} - W + 1$ .

## 10.4 RiskLSTM architecture and forward pass

```

1 n_feature_channels = 4 * n_assets + (3 if use_cross_sectional else 0)
2 self.lstm = nn.LSTM(input_size=n_feature_channels, hidden_size=hidden_size, ...)
3 if noisy_head:
4     self.head = NoisyLinear(hidden_size + n_assets, hidden_size // 2)
5     self.head_2 = NoisyLinear(hidden_size // 2, n_assets)
6 else:
7     self.head = nn.Linear(hidden_size + n_assets, hidden_size // 2)
8     self.head_2 = nn.Linear(hidden_size // 2, n_assets)
9
10 def forward(self, features, portfolio_weights):
11     lstm_out, (h_n, _) = self.lstm(features)
12     pooled = self.temporal_pool(lstm_out) if self.pooling == "attention" else h_n[-1]
13     hidden = self.dropout(pooled)
14     combined = torch.cat([hidden, portfolio_weights], dim=-1) # (B, H_r + N)
15     x = self.leaky_relu(self.head(combined)) # (B, H_r // 2)
16     attenuation_logit = torch.sigmoid(self.head_2(x)) # (B, N)
17     return self.max_attenuation + (1.0 - self.max_attenuation) * attenuation_logit

```

Listing 21: `RiskLSTM.__init__` and `forward`.

models/risk\_lstm.py:349-441

$$a_i = \text{max\_attenuation} + (1 - \text{max\_attenuation}) \cdot \sigma(\text{Head}_2(\text{LeakyReLU}(\text{Head}([\text{pooled}_i; \hat{w}_i]))) \quad (42)$$

one value per asset,  $a_i \in [\text{max\_attenuation}, 1]^N$  (default floor 0.33: an asset is de-risked, **never fully zeroed out**). The final position is  $w_i^{\text{final}} = \hat{w}_i \odot a_i$  (elementwise).

**Rationale.** The weight vector re-enters at the *head*, not the LSTM’s input, mirroring `PortfolioLSTM`’s `use_prev_weight` conditioning: the LSTM branch tells the network whether this asset’s *recent behavior* looks stable, while the concatenated weight tells it *what position is actually being considered* – both are needed to decide how much to de-risk. LeakyReLU (not plain ReLU) between `head` and `head_2` keeps a small gradient flowing even for units with a negative pre-activation, avoiding “dying ReLU” units that get stuck early in training and never update again. `max_attenuation` is a **floor**, **not a ceiling**: the smooth rescaling  $\text{max\_att} + (1 - \text{max\_att})\sigma(\cdot)$  has no dead zone (unlike a hard clamp), and the network can never force a position fully to zero regardless of how unstable an asset looks – a deliberate risk limit on the overlay’s own authority.

## 10.5 Risk training loop

```

1  attenuation = risk_model(noisy_features_train, portfolio_weights_train) # (B, N) - weights
   FIXED input
2  scaled_weights = portfolio_weights_train * attenuation
3  portfolio_returns = apply_transaction_costs_torch(scaled_weights, (scaled_weights *
   next_returns_train).sum(-1), transaction_cost)
4  loss = -sharpe_ratio(portfolio_returns) # plain whole-period Sharpe (not rolling_window_ratio)
5  loss.backward(); optimizer.step()

```

Listing 22: train\_risk\_model: one epoch.

models/risk\_lstm.py:648-750

**Rationale.** `portfolio_weights_train` is a fixed, already-detached input – gradients here only ever update RiskLSTM’s own parameters, never PortfolioLSTM’s. Unlike the portfolio training loop, there is **no best-checkpoint selection inside this loop**: `add_risk_overlay`’s restart selection (§12) picks the best restart *post-hoc* by validation Sharpe instead, since a single restart’s own epoch-by-epoch validation curve is far noisier for this smaller network.

## 11 Inverse-Volatility Benchmark

$$b_t^{\text{raw}} = \sum_{k=1}^N \frac{1/\sigma_t^{(k)}}{\sum_{j=1}^N 1/\sigma_t^{(j)}} r_{t+1}^{(k)}, \quad \sigma_t^{(k)} = \text{std}(X_t^{(\cdot,k)}) + 10^{-8} \quad (43)$$

models/portfolio\_lstm.py:1460-1485 (inverse\_vol\_benchmark\_returns)

An un-learned, classic inverse-volatility (“naive risk-parity”) allocator: weight each asset inversely to its own trailing realized volatility over that day’s *same* lookback window the model itself saw, normalized to sum to 1 – no vol-targeting leverage, no transaction-cost awareness.

$$b_t = b_t^{\text{raw}} \cdot \frac{\widehat{\text{std}}(r^{\text{model}}) \cdot \sqrt{252}}{\widehat{\text{std}}(b^{\text{raw}}) \cdot \sqrt{252}} \quad (44)$$

models/portfolio\_lstm.py:1488-1507 (vol\_match\_benchmark)

rescaled by **one constant multiplier per split** so its realized annualized volatility matches the *model’s own*, computed **separately for train, validation, and test**.

**Rationale.** A plain risk-parity book runs at whatever volatility the market happens to produce, while the model is actively vol-targeted – comparing their raw PnL would conflate “did the learned allocation add value” with “did the model just happen to run hotter or colder than the benchmark this period.” Rescaling the benchmark to the model’s own realized volatility (per split, since realized volatility differs a lot between train/val/test) isolates the former question, which is the one an allocation-skill comparison actually needs answered.

## 12 Ensembling: Multi-Seed Restarts

**Optional component (opt-in, default off).** `n_seeds > 1` (default 1) trains that many independently-seeded restarts of PortfolioLSTM (and, if enabled, RiskLSTM) on identical data, combined via `restart_strategy`.

```

1  if args.restart_strategy == "best":
2      best_idx, best = max(enumerate(results), key=lambda item: float(sharpe_ratio(item[1].
   returns_val)))
3      return best
4  # "ensemble":
5  ensemble_model = EnsemblePortfolioLSTM([r.model for r in results])

```

```
6 return evaluate_portfolio_model(ensemble_model, data, ...)
```

Listing 23: "best" vs. "ensemble" restart combination.

models/portfolio\_lstm.py:2374-2425 (run\_pipeline\_multi\_seed)

$$\text{"best"} : \quad \text{seed}^* = \arg \max_s \text{SharpeRatio}(r_s^{\text{val}}) \quad (45)$$

$$\text{"ensemble"} : \quad \hat{w}_i = \frac{\bar{w}_i}{\|\bar{w}_i\|_1 + \epsilon}, \quad \bar{w}_i = \frac{1}{S} \sum_{s=1}^S \hat{w}_i^{(s)} \quad (46)$$

models/portfolio\_lstm.py:918-932 (EnsemblePortfolioLSTM.forward)

**Rationale.** The Sharpe-ratio training objective is non-convex in the LSTM's parameters (LSTM/softmax nonlinearities), so different random seeds converge to different local optima – exactly the problem ensembling addresses. **"best"** picks the single restart with the highest validation Sharpe; **"ensemble"** averages every restart's predicted weights (a simple average of several points on the simplex already sums to 1 for **softmax** – the renormalization is a no-op there – but is *not* a no-op for **tanh\_norm**, where individual models can disagree on an asset's sign, so re-normalizing restores the same "fully invested" invariant each individual model already satisfies). Averaging tends to cancel out each individual restart's idiosyncratic overfitting to its own local optimum. **EnsembleRiskLSTM** is simpler still: attenuation values already live in `[max_attenuation, 1]`, and a plain average of values in that range stays in that range, so no renormalization is needed there.

## 13 Configuration Reference

The full `DEFAULT_CONFIG` dictionary (models/portfolio\_lstm.py, lines 1820–1980) is the single source of truth for every default; a JSON config file or the `TrainRequest` POST body (api/server.py, lines 235–276) need only specify keys that *differ* from these defaults.

| Key             | Default   | Meaning                                        |
|-----------------|-----------|------------------------------------------------|
| lookback        | 30        | Sequence length $L$ (days)                     |
| years           | 8         | Years of price history to fetch                |
| train_frac      | 0.8       | Train fraction of the non-test remainder       |
| test_frac       | 0.1       | Held-out, most-recent test fraction            |
| weight_scheme   | softmax   | softmax (long-only) or tanh_norm (long/short)  |
| hidden_size     | 32        | PortfolioLSTM hidden width $H$                 |
| epochs          | 300       | Training epochs                                |
| lr              | $10^{-3}$ | Adam learning rate                             |
| dropout         | 0.1       | Dropout on the pooled hidden state             |
| weight_decay    | $10^{-4}$ | Adam L2 penalty                                |
| noise_std       | 0.05      | Input-window Gaussian noise std (train only)   |
| target_vol      | 0.20      | Annualized volatility target                   |
| max_leverage    | 10.0      | Cap on the vol-targeting scale factor          |
| noisy_head      | False     | NoisyNet parameter noise on the output head(s) |
| use_prev_weight | False     | Recurrent-policy feedback of $w_{i-1}$         |
| has_cash        | False     | Add a CASH pseudo-asset                        |
| cash_return     | 0.0       | CASH's constant daily return                   |
| use_carry       | False     | Add FX carry as an input channel               |
| vol_horizons    | []        | Vol-normalized-momentum horizons (days)        |
| encoder_type    | concat    | concat or per_asset                            |

| Key                  | Default   | Meaning                                         |
|----------------------|-----------|-------------------------------------------------|
| asset_combiner       | attention | attention or mean (per-asset encoder only)      |
| n_attn_heads         | 2         | Cross-asset attention heads                     |
| covariance_estimator | sample    | sample, ewma, or ledoit_wolf                    |
| ewma_lambda          | 0.94      | EWMA decay ( $\approx 11$ -day half-life)       |
| pooling              | last      | last ( $h_n[-1]$ ) or attention (both networks) |
| objective            | sharpe    | sharpe, kelly, or cvar                          |
| sharpe_window        | 60        | Rolling-window width for the "sharpe" objective |
| cvar_alpha           | 0.95      | CVaR confidence level                           |
| cvar_kappa           | 1.0       | CVaR risk-aversion weight                       |
| n_seeds              | 1         | Multi-restart count                             |
| restart_strategy     | best      | best or ensemble                                |
| risk_overlay         | False     | Enable the RiskLSTM stage                       |
| risk_hidden_size     | 16        | RiskLSTM hidden width $H_r$                     |
| risk_epochs          | 200       | RiskLSTM training epochs                        |
| risk_lr              | $10^{-3}$ | RiskLSTM Adam learning rate                     |
| max_attenuation      | 0.33      | Attenuation floor                               |
| risk_rolling_window  | 10        | $W$ : RiskLSTM's rolling-statistics window      |
| use_cross_sectional  | False     | Add the 3 cross-sectional risk channels         |
| transaction_cost     | 0.0       | Cost per unit turnover (bps)                    |
| load_portfolio       | / None    | Load a saved model instead of training          |
| load_risk            |           |                                                 |
| save_db              | False     | Persist to <code>quant.model_registry</code>    |

## 14 Summary of Architectural Decisions

| Decision                                                    | Primary rationale                                                                                                                                          |
|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sequential (not joint) two-network training                 | Keeps each network's gradient signal attributable to one well-posed sub-problem (asset selection vs. risk sizing).                                         |
| Log returns                                                 | Time-additive; matches the variance/covariance formulas used everywhere.                                                                                   |
| Carry & vol-normalized momentum (optional)                  | Standard, well-documented FX risk-adjusted signals, asset-major laid out for slice-ability.                                                                |
| Forward-fill only (never back-fill) for carry/vol features  | Any back-fill would leak a later day's value into an earlier row – a genuine look-ahead bias.                                                              |
| Per-asset shared encoder + cross-asset attention (optional) | Permutation-equivariant and, in principle, universe-size invariant, unlike the concatenated encoder's fixed asset-indexed weight matrices.                 |
| Attention pooling over time (optional)                      | $h_n[-1]$ alone can lose mid-window information for longer lookbacks; a learned pool lets the network decide which days matter most.                       |
| use_prev_weight recurrent policy                            | Lets the model reason about whether a rebalance is worth its transaction cost; implemented as a detached (not full-BPTT) sequential loop for tractability. |
| NoisyNet parameter noise (optional)                         | A second, complementary form of regularization (in weight-space, not input-space) against overfitting one exact historical sequence.                       |

| Decision                                              | Primary rationale                                                                                                                                                          |
|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NoisyLinear<br>freeze/resample fix                    | Sequential <code>forward_sequence</code> would otherwise mutate shared noise buffers in place mid-graph, corrupting autograd; noise is now drawn once per sequential pass. |
| CASH as a data-layer<br>pseudo-asset                  | Reuses every downstream mechanism (vol-targeting, costs, reporting) with zero special-casing; a constant return has exactly zero variance.                                 |
| Three covariance estimators                           | Sample covariance is noisy/possibly singular on a short window; EWMA and Ledoit-Wolf are progressively more robust, principled remedies.                                   |
| <code>max_leverage</code> cap                         | Bounds the worst-case realized loss from a spuriously near-singular covariance estimate; fixes Kelly’s dead-gradient failure mode at its source.                           |
| Rolling-window Sortino, Kelly, and CVaR objectives    | Three different, well-understood risk/return trade-offs (non-convex ratio, convex log-wealth, convex tail-risk penalty).                                                   |
| Annualizing both mean-CVaR terms                      | Prevents CVaR’s larger raw magnitude from starving the mean-return gradient signal.                                                                                        |
| Transaction costs inside the training loss            | Trains the model to avoid costly, unnecessary rebalancing directly, not only as a reporting afterthought.                                                                  |
| Chronological, test-carved-first 3-way split          | Test never influences any training or model-selection decision – a genuinely unbiased generalization estimate, unlike validation.                                          |
| Best-val-Sharpe checkpoint selection                  | Targets the epoch that generalized best, not whichever the fixed epoch budget happened to land on.                                                                         |
| Marginal + cross-sectional risk features for RiskLSTM | Per-asset moments see individual risk; cross-sectional correlation/eigenvalue features see portfolio-wide concentration risk that marginal statistics alone cannot.        |
| Attenuation floor (never fully zero)                  | A deliberate limit on the overlay’s authority – it can de-risk, never fully close, a position.                                                                             |
| Multi-seed restarts & ensembling                      | Both training objectives are non-convex; restarts and averaging mitigate sensitivity to a single unlucky initialization.                                                   |
| Inverse-vol benchmark, vol-matched per split          | Isolates “did the learned allocation add value” from “did the model just run hotter/colder than a naive book.”                                                             |

## 15 Repository Map

| Path                                  | Contents                                                                                                                                                                                                                                                    |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>models/portfolio_lstm.py</code> | Data pipeline, <code>PortfolioLSTM/EnsemblePortfolioLSTM</code> , <code>NoisyLinear</code> , <code>TemporalAttentionPool</code> , volatility targeting, benchmark, training objectives, transaction costs, the full portfolio training/evaluation pipeline. |
| <code>models/risk_lstm.py</code>      | <code>RiskLSTM/EnsembleRiskLSTM</code> , rolling moments, cross-sectional features, risk-sequence construction, risk training/evaluation pipeline.                                                                                                          |



| Path                                        | Contents                                                                                                                                                                 |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>models/risk_postprocess.py</code>     | Plotting helpers for the risk-overlay pipeline (position/attenuation, vol-matched PnL, return histograms, transaction-cost PnL).                                         |
| <code>data/db.py</code>                     | Generic ( <code>symbol</code> , <code>source</code> , <code>field</code> )-keyed time-series storage/retrieval (prices and FRED rates share this schema).                |
| <code>data/fx_downloader.py</code>          | FX price downloader used to backfill missing history.                                                                                                                    |
| <code>data/rates_downloader.py</code>       | FRED interest-rate downloader (feeds <code>load_carry</code> ).                                                                                                          |
| <code>data/model_registry.py</code>         | Postgres-backed <code>quant.model_registry</code> model blob storage.                                                                                                    |
| <code>api/server.py</code>                  | FastAPI app: <code>TrainRequest</code> / <code>EvaluateRequest</code> schemas, background training jobs, live epoch-report callback, stop endpoint, evaluation endpoint. |
| <code>frontend/src/TrainingView.js</code>   | Training configuration form, live in-sample/validation/test PnL charts, final Sharpe table and PnL/benchmark charts.                                                     |
| <code>frontend/src/EvaluationView.js</code> | Loads a saved model (or pair) purely for inference/backtesting display.                                                                                                  |
| <code>main.py</code>                        | Single CLI entry point: reads a JSON config, dispatches to the training/evaluation pipeline functions above.                                                             |